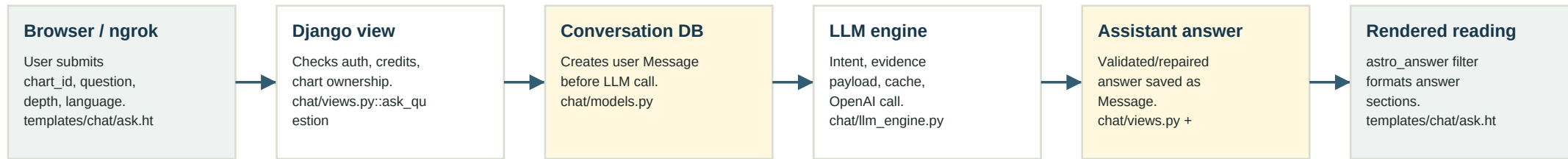
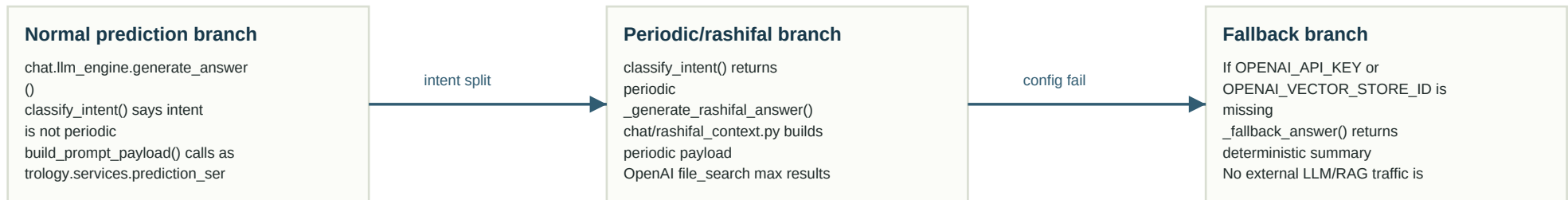


Astrology GPT: User Question Flow

Live Django request path, deterministic astrology pipeline, RAG/LLM traffic, validation, and storage.



High-level rule: Software calculates. Rules judge. RAG supports. LLM explains. Validator controls quality.

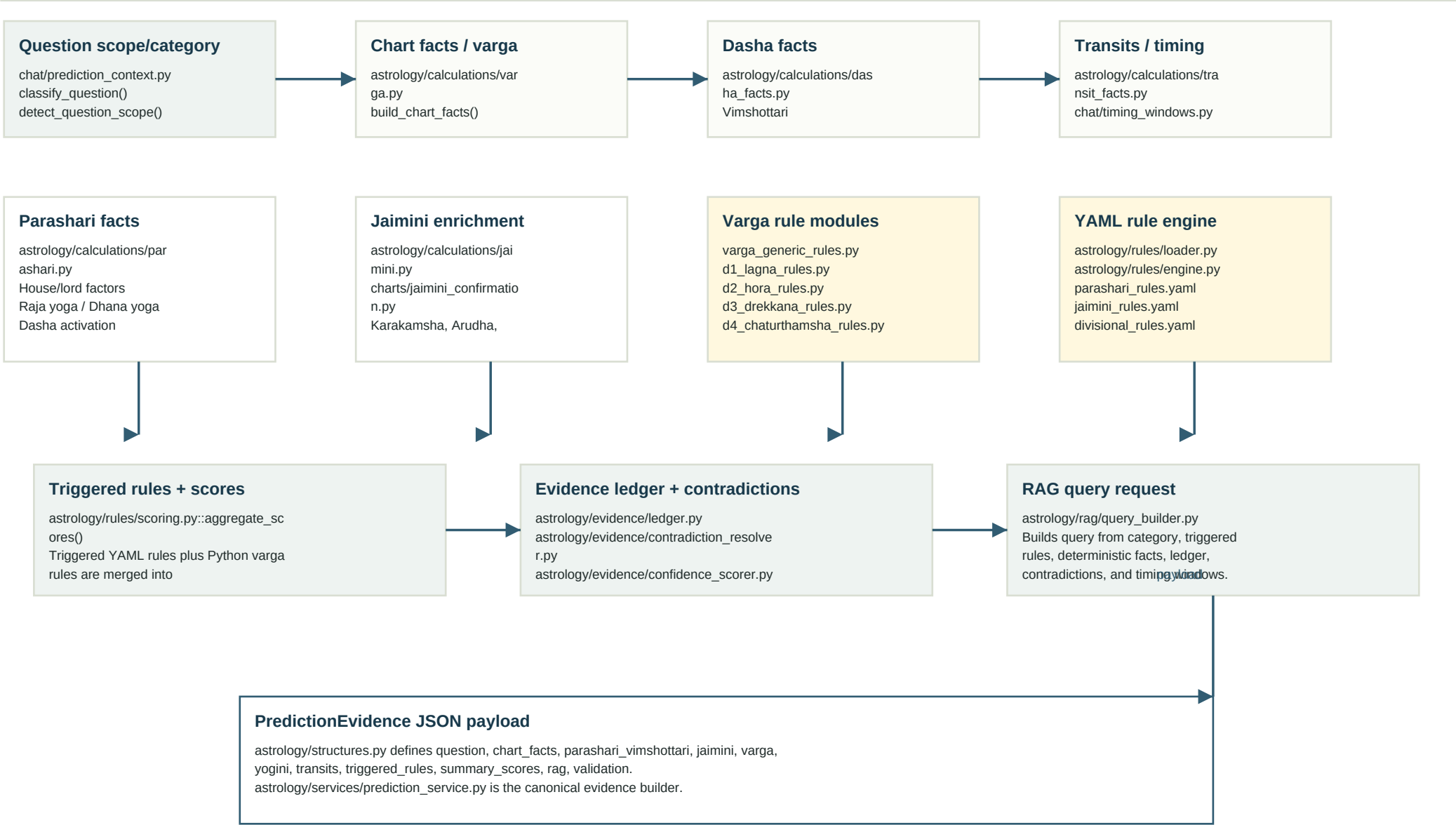


Important current boundary:

- The architecture package astrology/ owns deterministic evidence, rules, RAG query construction, synthesis payloads, and validation helpers.
- The live traffic orchestrator is still chat/llm_engine.py: it builds compact evidence, calls OpenAI, runs validation, repairs, applies deterministic remedies, caches, and returns the answer.
- SavedChart.chart_data is assumed to already contain chart calculations from charts/ code; prediction_service wraps and normalizes that data instead of recalculating from the LLM.

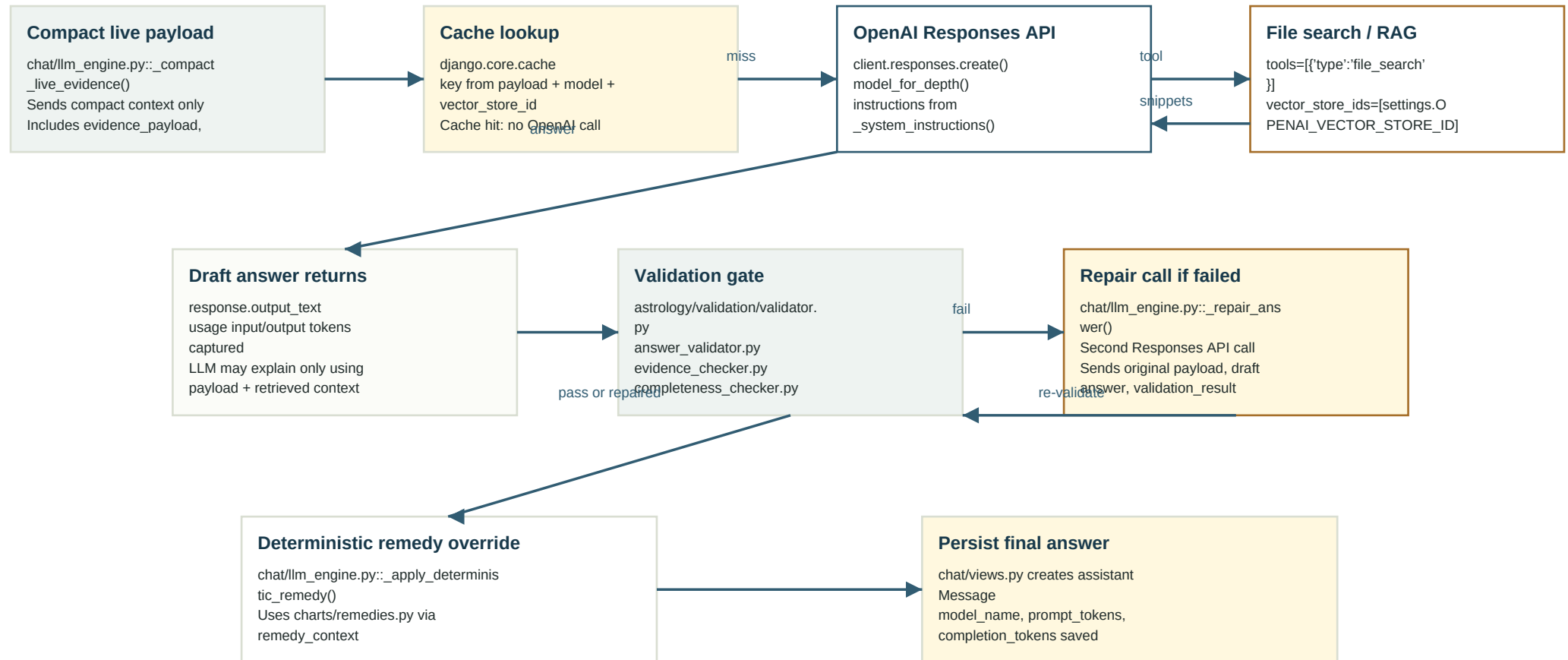
Deterministic Evidence Construction

Files used before any LLM synthesis. This is the core no-invention boundary.



LLM, RAG, Cache, Validation, and Repair Traffic

External network calls and the exact places where generated text is allowed.



Traffic summary

- Inbound user traffic: Browser or ngrok public URL -> Django route chat/urls.py path('ask/') -> chat.views.ask_question().
- Internal deterministic traffic: Python function calls over local chart_data, no LLM calculation.
- External LLM/RAG traffic: Django server -> OpenAI Responses API. File search is requested as an OpenAI tool against OPENAI_VECTOR_STORE_ID.
- Cache hit traffic: if the cache key exists, Django returns cached answer and no OpenAI/File Search request is made.
- Failure traffic: missing OPENAI_API_KEY or OPENAI_VECTOR_STORE_ID returns _fallback_answer(), no external call.

Evidence Payload and File Responsibility Map

What each part of the JSON means and which files own it.

Required JSON evidence shape - deterministic side

question: astrology/structures.py::QuestionContext; populated by prediction_service.py from chat/prediction_context.py.
chart_facts: astrology/calculations/varga.py wraps SavedChart.chart_data; source chart generation is in charts/parashari and parashari_vimshottari: parashari.py + dasha_facts.py.
jaimini: dasha_facts.py + jaimini.py + charts/jaimini_confirmation.py.
varga: varga.py plus Python varga rule modules such as d7_saptamsha_rules.py.
yogini: astrology/calculations/yogini.py and charts/yogini_alignment.py for timing windows.
transits: transit_facts.py + chat/timing_windows.py + charts/transit_priority.py.

Required JSON evidence shape - judgment/support side

triggered_rules: astrology/rules/engine.py YAML rules plus Python deterministic varga rules.
summary_scores: astrology/rules/scoring.py aggregates triggered rule outcomes.
evidence_ledger: astrology/evidence/ledger.py creates traceable claims.
contradictions: astrology/evidence/contradiction_resolver.py flags mixed signals.
confidence_summary: astrology/evidence/confidence_scorer.py summarizes evidence strength.
rag: astrology/rag/query_builder.py builds search terms; OpenAI file_search retrieves source snippets at call time.
validation: astrology/validation/* checks final answer and drives repair_instruction.

Strict LLM contract

- The LLM must not calculate placements, houses, dashas, vargas, yogas, transits, or rule applicability.
- The LLM receives already-calculated evidence and may synthesize, explain, and cite retrieved support.
- RAG is not the rule engine. RAG supports explanation after deterministic facts and triggered rules exist.
- Validation can force a repair pass when the answer is generic, unsupported, missing systems, or contradicts evidence.

Notable live files in request order

- templates/chat/ask.html -> chat/views.py -> chat/models.py.
- chat/llm_engine.py -> astrology/services/prediction_service.py.
- prediction_service.py -> calculations, rules, evidence, rag, synthesis.
- chat/llm_engine.py -> OpenAI Responses API with file_search vector store.
- astrology/validation/validator.py -> repair call when failed.
- chat/llm_engine.py::_apply_deterministic_remedy() -> chat/views.py saves final Message.
- templates/chat/ask.html + chat/templatetags/chat_formatting.py render final reading.

Current architecture note

The deterministic architecture is centered in astrology/. The live LLM adapter is still chat/llm_engine.py, which is the main future refactor target.

Generated file: astrology/docs/user_question_flow.pdf